

validation_chain_order

validation chain order — hash-before-signature (G2 resolution)

field	value
document	validation_chain_order.md
purpose	Resolve gap G2 (docs/research/main_specs/research/additions_synthesis.md §8): verify and document the exact S2 (SHA-256 hash) vs S3 (ed25519 signature) ordering in the ota-artifact-validator brick.
status	FACT — verified against brick source; G2 SATISFIED, no code change required.
verified order	S2 (hash) BEFORE S3 (signature).
evidence basis	ota-artifact-validator Go source (read-only) + server consumer handler. No .go source was modified.
date	2026-06-08
author	security code audit (read-only)

table of contents

- [summary](#)
- [verified stage order](#)
- [what each stage verifies](#)
- [signature binds to the hashed bytes](#)
- [fail fast proof from tests](#)
- [http mapping](#)
- [why order matters](#)
- [g2 verdict](#)
- [unverified residue](#)

summary

The ota-artifact-validator brick runs a fail-fast, ordered pipeline $S2 \rightarrow S3 \rightarrow S4 \rightarrow S5 \rightarrow S6$. **S2 (SHA-256 hash check) runs strictly before S3 (ed25519 signature check)**, and S3 verifies the detached signature over the *digest produced by S2* — not over the raw artifact bytes independently. This is the safety-correct order. G2 is SATISFIED as FACT with no required change to the brick.

verified_stage_order

The order is fixed in the top-level Validate function:

- submodules/ota-artifact-validator/pipeline.go:63–70 — **S2** (ValidateHash) runs first; on reject it returns immediately (fail-fast), so S3 never runs.
- submodules/ota-artifact-validator/pipeline.go:72–78 — **S3** (ValidateSignature) runs second, only if S2 passed.
- submodules/ota-artifact-validator/pipeline.go:80–100 — S4 (version), S5 (target), S6 (metadata) follow, each fail-fast.

Stage identifiers are stable constants: - submodules/ota-artifact-validator/verdict.go:26 — StageHash Stage = "S2". - submodules/ota-artifact-validator/verdict.go:28 — StageSignature Stage = "S3".

Exact code (pipeline.go:60–78):

```
func Validate(in Input) Result {
    var res Result

    // S2 - SHA-256 vs hash file.
    hashVerdict, digest := ValidateHash(in.Artifact,
        in.HashFile) // pipeline.go:64
    res.ComputedSHA256 = digest
    res.Verdicts = append(res.Verdicts, hashVerdict)
    if hashVerdict.IsReject() {
        res.Final = hashVerdict
        return res // pipeline.go:69 - fail-fast: S3 not reached
    }

    // S3 - detached signature over the S2 digest.
    sigVerdict := ValidateSignature(digest, in.PublicKey,
        in.Signature) // pipeline.go:73
    res.Verdicts = append(res.Verdicts, sigVerdict)
    if sigVerdict.IsReject() {
        res.Final = sigVerdict
        return res
    }
    ...
}
```

what_each_stage_verifies

S2 — ValidateHash (submodules/ota-artifact-validator/stages.go:47–66): - Rejects an empty/absent hash file (stages.go:48–49, RejectHashFileMissing). - Rejects a hash file lacking a valid 64-char lowercase-hex SHA-256 (stages.go:51–54, RejectHashFileMalformed). - Streams the artifact bytes through sha256.New() and io.Copy (stages.go:56–60). - Compares the computed lowercase-hex digest to the expected value (stages.go:62–64); mismatch → RejectHashMismatch. - On success returns the computed digest string for downstream use (stages.go:65).

S3 — ValidateSignature (submodules/ota-artifact-validator/stages.go:93–108): - Rejects a missing signature (stages.go:94–96, RejectSignatureMissing). - Rejects a public key that is not a valid ed25519 key length (stages.go:97–99, RejectSignatureKeyInvalid). - Decodes the hex digest and rejects a malformed/wrong-length digest (stages.go:100–103, RejectSignatureScopeMismatch). - Verifies the detached ed25519 signature **over the digest** via ed25519.Verify(pubKey, digest, sig) (stages.go:104); failure → RejectSignatureInvalid.

signature_binds_to_the_hashed_bytes

S3 does not re-read or re-verify the raw artifact. It verifies the signature over the *exact digest S2 computed and returned*:

- pipeline.go:64 — ValidateHash(...) returns digest.
- pipeline.go:73 — that same digest is passed into ValidateSignature(digest, ...).
- stages.go:104 — ed25519.Verify(pubKey, digest, sig) verifies over that digest.

Doc-comment confirmation in source (stages.go:84–87): “*digestHex is the lowercase-hex SHA-256 produced by S2 (this binds the signed bytes to the same artifact that passed S2 — the scope-match requirement).*” And stages.go:91–92: the signature is over the digest, not the raw blob, keeping verification streaming-friendly.

This means the signature scope is the S2 digest, and S2 has already proven that digest equals the actual artifact bytes. The two stages are therefore coupled: passing S2 establishes “these bytes hash to D”, and S3 establishes “D is signed by the trusted key”.

fail_fast_proof_from_tests

The ordering is not only structural but asserted by the brick’s own tests (read-only evidence; tests were not run as part of this audit):

- submodules/ota-artifact-validator/validator_test.go:525–526 — asserts exactly two verdicts (S2, S3) run when S3 rejects.

- `submodules/ota-artifact-validator/validator_test.go:528-529` — asserts `Verdicts[0].Stage == StageHash` and `Verdicts[1].Stage == StageSignature`, i.e. S2 precedes S3.
- `validator_test.go:99-117` — hash-stage reject cases expect `wantStage: StageHash`.
- `validator_test.go:126-145` — signature-stage reject cases expect `wantStage: StageSignature`.

http_mapping

The server consumes the brick in `server/internal/api/handlers_artifact.go`:

- `handlers_artifact.go:159` — calls `otavalidator.Validate(in)`.
- `handlers_artifact.go:160-162` — on any reject, calls `respondValidatorReject(c, result.Final)` with the first (decisive) failing verdict.
- `respondValidatorReject` (`handlers_artifact.go:201-219`) maps stage → HTTP:
 - `handlers_artifact.go:203-205` — StageHash (S2) → **422 HASH_MISMATCH** (`CodeHashMismatch`).
 - `handlers_artifact.go:206-208` — StageSignature (S3) → **422 SIGNATURE_INVALID** (`CodeSignatureInvalid`).
 - `handlers_artifact.go:209-211` — StageVersion (S4) → 409 `VERSION_NOT_MONOTONIC`.
 - `handlers_artifact.go:212-213` — StageTarget (S5) → 400 `VALIDATION_FAILED`.
 - `handlers_artifact.go:214-215` — StageMetadata (S6) → 400 `VALIDATION_FAILED`.

Because the pipeline is fail-fast and `result.Final` holds the *first* rejecting verdict, a hash-mismatch artifact deterministically yields **422 HASH_MISMATCH** and never reaches the signature check — matching the claim in G2.

Related trust-boundary note (already enforced, cross-referenced for completeness): the S3 verification key is taken **only** from server config via `resolvePublicKey` (`handlers_artifact.go:264-269`), never from the request (`handlers_artifact.go:35-37`, `handlers_artifact.go:256-263`).

why_order_matters

Verifying the hash before the signature is the safety-correct ordering, for two reasons specific to how this brick is built:

1. **The signature's scope is the digest.** S3 verifies a signature over the SHA-256 digest, not over the raw bytes. If S3 ran first (or alone), a passing signature would only prove “the trusted key signed digest D” — it would say nothing about whether the artifact bytes on hand actually hash to D. S2 is what binds D to the concrete bytes. Running S2 first guarantees that by the time S3 verifies the signed digest, that digest is already proven to equal the artifact. Running

them in the reverse order would leave a window where a validly-signed digest is accepted while the accompanying bytes are arbitrary (a payload-substitution / hash-substitution gap).

2. **Cheaper, attacker-independent rejection first.** S2 is a pure self-consistency check (bytes vs. their declared hash) that needs no secret/trusted material and rejects corrupt or substituted payloads before any key-dependent cryptographic verification. This is defense-in-depth ordering: reject the obviously-broken artifact on a deterministic content check before spending the signature verification, and before any reject message could differ based on key material.

The combined invariant the order produces: **accept** $\iff$ (**bytes hash to D**) $\wedge$ (**D is signed by the trusted key**) — and S6 additionally cross-checks `meta.SHA256 == D` (`stages.go:172–173`), closing the loop between declared metadata and the verified digest.

g2_verdict

G2 is SATISFIED as FACT. The brick verifies S2 (hash) strictly before S3 (signature) — `pipeline.go:64` then `pipeline.go:73` with fail-fast at `pipeline.go:69` — and S3 verifies the signature over the S2 digest (`pipeline.go:73`, `stages.go:104`). The order is additionally pinned by `validator_test.go:528–529`. The server maps S2→422 HASH_MISMATCH and S3→422 SIGNATURE_INVALID (`handlers_artifact.go:203–208`), consistent with the G2 statement. **No code change is required in the brick.**

unverified_residue

- **Tests not executed.** Ordering is cited from source structure and from the assertions in `validator_test.go`; this audit was read-only and did not run `go test`. The assertions themselves are evidence, but a green test run was not observed in this pass.
- **Spec-text alignment.** G2's remediation asks that the order also be stated in `security/signing_verification.md`. This document records the verified fact; whether `signing_verification.md` has been updated with the explicit S2-before-S3 sentence was not confirmed here and remains a separate documentation task.
- **S1 (structure) is outside the brick.** S1 ZIP_STORED parsing runs in the handler (`handlers_artifact.go:103–113`, `validateStructure` at :234) before S2; it was reviewed only insofar as it precedes the S2/S3 pair and does not affect their relative order.